

VIT[®]
BHOPAL

**School of Computing Science Engineering &
Artificial Intelligence
(SCAI)**

AI & ML Record File

Fundamentals in AI & ML(CSA2001)

Slot: C11+C12+C13

Class No: 0811

Submitted By:

Name- Shivanand Irappa Sangammanavar

Reg. No.: 25BAI11346

Submitted To:

Dr. Monica Sankat

Senior Assistant

Professor, SCOPE

Smart Traffic Pollution Reducer

AIM

To develop a Machine Learning-based system to predict traffic congestion and estimate pollution levels, and to optimize traffic signal timing for reducing emissions

PROBLEM STATEMENT

Traffic congestion in urban areas leads to increased fuel consumption, higher CO₂ emissions, and longer waiting times. Traditional traffic systems are static and do not adapt to real-time conditions, resulting in inefficiency and environmental impact.

OBJECTIVE

- To predict traffic congestion using input parameters
- To estimate pollution levels based on congestion
- To suggest optimized traffic signal timings

TOOLS AND TECHNOLOGIES

- Python
- Pandas
- NumPy
- Scikit-learn (Random Forest)
- Matplotlib

PROCEDURE

1. Create a project folder and organize files into proper structure.
2. Generate synthetic traffic data including hour, day, vehicle count, and temperature.
3. Preprocess the dataset and define input features and target variable.

4. Train a Random Forest Regression model on the dataset.
5. Take user input for traffic parameters.
6. Predict traffic congestion using the trained model.
7. Estimate pollution level based on congestion.
8. Apply rule-based logic to optimize signal timing.
9. Display the results and visualize them using a graph.

FLOW DIAGRAM

SYSTEM FLOW DIAGRAM

```
Start
↓
Input Traffic Data (hour, day, vehicles, temperature)
↓
Generate / Load Dataset
↓
Train Machine Learning Model (Random Forest)
↓
Take User Input
↓
Predict Traffic Congestion
↓
Estimate Pollution Level
↓
Apply Signal Optimization Logic
↓
Display Output (congestion, CO2, signal timing)
↓
Show Graph Visualization
↓
End
```

ALGORITHM

1. Start
2. Generate dataset
3. Train Random Forest model
4. Accept user input
5. Predict congestion
6. Calculate pollution
7. Optimize signal timing
8. Display results
9. Stop

CODE

```
traffic_system.py - C:\Users\vishn\OneDrive\Documents\Smart-Traffic-Pollution-Reducer\src\traffic_system.py (3.12.0)
File Edit Format Run Options Window Help

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt

np.random.seed(42)

n_samples = 200

data = pd.DataFrame({
    "hour": np.random.randint(0, 24, n_samples),
    "day": np.random.randint(0, 7, n_samples),
    "vehicles": np.random.randint(50, 500, n_samples),
    "temperature": np.random.randint(20, 40, n_samples)
})

data["congestion"] = (
    data["vehicles"] * 0.15 +
    (data["hour"].apply(lambda x: 1 if 8 <= x <= 10 or 17 <= x <= 19 else 0) * 50) +
    np.random.randint(0, 20, n_samples)
)

data["congestion"] = data["congestion"].clip(0, 100)

X = data[["hour", "day", "vehicles", "temperature"]]
y = data["congestion"]
```

```

model = RandomForestRegressor()
model.fit(X, y)

def predict_traffic(hour, day, vehicles, temperature):
    input_data = pd.DataFrame([[hour, day, vehicles, temperature]],
                               columns=["hour", "day", "vehicles", "temperature"])
    return model.predict(input_data)[0]

def estimate_pollution(congestion):
    return congestion * 0.5

def optimize_signal(congestion):
    if congestion > 70:
        return "Increase Green Time (60s)"
    elif congestion > 40:
        return "Moderate Green Time (40s)"
    else:
        return "Reduce Green Time (20s)"

print("\n🚦 Smart Traffic Pollution Reducer\n")

hour = int(input("Enter hour (0-23): "))
day = int(input("Enter day (0=Mon ... 6=Sun): "))
vehicles = int(input("Enter number of vehicles: "))
temp = int(input("Enter temperature: "))

predicted_congestion = predict_traffic(hour, day, vehicles, temp)
pollution = estimate_pollution(predicted_congestion)
signal = optimize_signal(predicted_congestion)

```

```

print(f"\n📊 Predicted Congestion: {predicted_congestion:.2f}%")
print(f"🌫️ Estimated CO₂ Emission: {pollution:.2f} kg/h")
print(f"📶 Signal Recommendation: {signal}")

plt.bar(["Congestion", "CO2"], [predicted_congestion, pollution])
plt.title("Traffic vs Pollution")
plt.show()

```

INPUT

Example:

- Hour: 9
- Day: 1
- Vehicles: 300
- Temperature: 30

```

C:\Users\vishn\OneDrive\Documents\Smart-Traffic-Pollution-Reducer>python src/traffic_system.py

🚦 Smart Traffic Pollution Reducer
9
Enter day (0=Mon ... 6=Sun): 1
Enter number of vehicles: 300
Enter temperature: 30

```

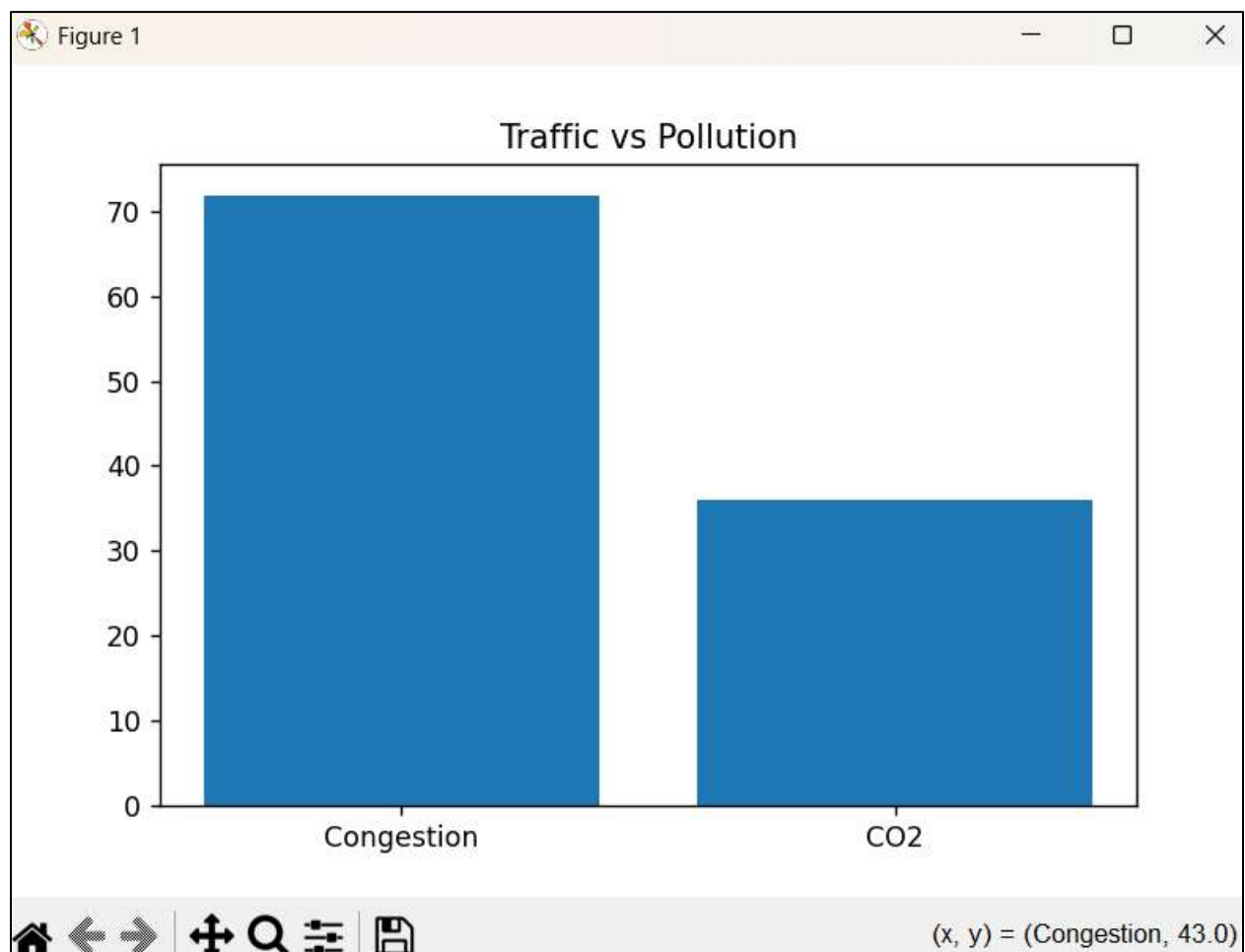
OUTPUT

- Predicted Congestion: 71.86%
- Estimated CO₂ Emission: 35.93 kg/h
- Signal Recommendation: Increase Green Time (60s)

```
Predicted Congestion: 71.86%  
Estimated CO2 Emission: 35.93 kg/h  
Signal Recommendation: Increase Green Time (60s)
```

RESULT

The system successfully predicts traffic congestion and estimates pollution levels. It also suggests optimized signal timings, demonstrating how adaptive traffic control can reduce emissions and improve traffic flow.



CONCLUSION

This project shows how Machine Learning can be applied to solve real-world problems such as traffic congestion and pollution. The system provides a simple yet effective approach to improving traffic management.

APPLICATIONS

- Smart city traffic management
- Pollution control systems
- Urban planning and transportation systems

LIMITATIONS

- Uses synthetic data instead of real-world data
- Pollution estimation is based on a simple formula
- Does not handle real-time traffic updates

FUTURE SCOPE

- Integration with real-time traffic data
- Use of advanced ML models like LSTM
- Deployment as a web or mobile application
- Integration with IoT sensors

REFERENCES

- Scikit-learn Documentation
- Python Official Documentation
- Machine Learning Fundamentals

GITHUB LINK

<https://github.com/shivanand25bai11346/Smart-Traffic-Pollution-Reducer>